KOR STRUCTURAL · IT INFRASTRUCTURE

WorkstationOps Toolkit

Remote diagnostics and remediation for the KOR workstation fleet — built for a network where WinRM and RPC are blocked, so the usual remote-management tooling simply does not work.

CONTENTS

OVERVIEW

What it is

A PowerShell module that diagnoses and repairs Windows workstations over the network, without needing anything installed on the target machine and without WinRM.

It answers one question well — *what is actually wrong with this machine?* — and then fixes the subset of problems that can be fixed remotely. One call replaces what was, on the night it was written, roughly forty ad-hoc commands.

```
Import-Module .\tools\WorkstationOps\Kor.WorkstationOps.psd1
Get-KorWorkstationHealth -ComputerName KOR-206-N | Select-Object -ExpandPropert
```

Location	`tools/WorkstationOps/`	**Files**	`Kor.WorkstationOps.psm1` + `.psd1`
Requires	PowerShell 7, domain admin rights	**Installs**	nothing on the target

ORIGIN

Why it exists

It came out of a single helpdesk ticket — Kevin's Outlook calendar was slow and crashing. Answering that properly meant pulling event logs, add-in inventories, Office build versions, mailbox store sizes and crash signatures off a dozen machines by hand.

Two things became obvious. First, the same handful of questions get asked of every sick machine. Second — and this is the part that justified building a

module rather than a script — **a single machine's symptoms mean very little without the fleet as a control group.**

> THE PATTERN WORTH KEEPING
>
> Kevin's 12 GB mailbox looked like an obvious cause until the fleet sweep showed another user running 45 GB without complaint. A six-month-old graphics driver looked like an obvious cause until a machine with the identical driver showed zero crashes. **Nearly every plausible single-machine explanation died against the control group.** The one that survived — the Access Database Engine conflict — did so on an 8/8 clean split.

So the module is built to be run across many machines cheaply, and to report findings in a form you can compare.

CONSTRAINTS

The three channels

Standard remote management is unavailable on this network. That is not a bug to fix before proceeding — it is the environment the tool has to work in.

CHANNEL	STATE	WHAT IT GIVES
c$ / ADMIN$ (445)	works	read + write file access, event-log extraction, folder operations
sc.exe \\host	works	service stop/start via the svcctl named pipe
reg.exe \\host	conditional	registry — but only while RemoteRegistry runs, and it ships Disabled
WinRM 5985/5986	blocked	—
RPC dynamic ports	blocked	— no `Get-WinEvent -ComputerName`, no `Get-CimInstance`, no `schtasks /s`

Every function is built on the first three. The event-log workaround is the clearest example: since logs cannot be queried remotely, the module copies the `.evtx` file over `c$` and parses it locally with `Get-WinEvent -Path`. Slower, but full fidelity — including provider-specific events like Outlook's.

> REMOTEREGISTRY DISCIPLINE
>
> RemoteRegistry is Disabled by default, and that is the correct posture given two confirmed mailbox compromises on the tenant. `Use-KorRemoteRegistry` starts it, runs your scriptblock, and restores **both** the original start type and run state in a `finally` block — so an exception mid-body cannot leave the service exposed. Never call `reg.exe` against a workstation outside that wrapper.

CONCEPTS

What the catalog is

The Windows Search catalog is the index database — the thing that makes Outlook's search box return results instantly instead of asking the server.

It lives in one folder, and the module reads its size as a progress signal:

```
C:\ProgramData\Microsoft\Search\Data\Applications\Windows\
```

FILE	WHAT IT HOLDS
`Windows.db`	The index itself — full-text content and a property store for every indexed item. This is the bulk of the catalog.
`Windows-gather.db`	The gatherer's work queue: what has been crawled, what is still pending.
`Windows-usn.db`	Tracks the NTFS change journal so edits are picked up incrementally rather than by re-crawling everything.
`*-wal / *-shm`	SQLite write-ahead log and shared-memory files. Windows 11 moved Search from the older ESE engine (`Windows.edb`) to SQLite.

Why Outlook cares

Outlook's instant search does not scan the mailbox — it queries this catalog. Crucially, **the OST is one of the indexed locations**, so a user's entire mailbox is represented in here. When the catalog is incomplete or corrupt, Outlook returns partial results, or falls back to a much slower server-side search, and the machine feels sluggish while the indexer thrashes trying to recover.

That failure announces itself as **Outlook event 36** — "*Search cannot complete the indexing of your Outlook data*" — which is one of the specific things `Get-KorWorkstationHealth` looks for.

Why size is the progress signal

Catalog size is roughly proportional to how much content has been indexed. A rebuild wipes it to near zero and it regrows over hours. That gives a cheap, reliable way to confirm a rebuild actually started — and to confirm it is still running — without any agent on the machine.

MEASURED 2026-07-28

- **KOR-206-N** — 1014 MB → 4.8 MB at wipe → 306.5 MB and climbing
- **KOR-207** — 339.9 MB → 16.2 MB at wipe → 69.7 MB and climbing

THE THROTTLE THAT LOOKS LIKE A FAULT

Windows Search **backs off hard while a user session is active.** A catalog that shows zero growth across several samples during the working day is almost certainly being throttled, not stuck. On KOR-206-N the index sat motionless for over an hour, then resumed at roughly 60 MB per twelve minutes the moment the user logged off. Do not diagnose a stalled rebuild without first checking whether anyone is on the machine.

REFERENCE

Function reference

Diagnostics — read-only, safe to run anytime

FUNCTION	WHAT IT DOES
`Get-KorWorkstationHealth`	The orchestrator. Runs everything below and returns a ranked `Findings` array. Accepts pipeline input for fleet sweeps.
`Test-KorWorkstationChannel`	Probes which management channels are open before assuming any of them. Run this first against an unfamiliar machine.
`Get-KorWorkstationEvent`	Copies an event log over `c$` and parses it locally. Caches per machine; `-Refresh` re-pulls.
`Get-KorOutlookAddin`	Reconstructs the add-in inventory *with per-add-in load times* from Outlook event 45, plus any add-in Outlook disabled for causing a crash (event 59).
`Get-KorOutlookStore`	Every OST/NST per profile, with size, age, duplicate detection and index-error attribution.
`Get-KorOfficeHealth`	Office build, and the Access Database Engine shared-DLL conflict described below.

Remediation — changes state, supports `-WhatIf`

FUNCTION	WHAT IT DOES
`Set-KorOutlookAddinState`	Sets an add-in's LoadBehavior and **returns the prior value plus a ready-to-paste rollback command.** Takes several ProgIDs at once so they share one RemoteRegistry cycle.
`Invoke-KorSearchIndexRebuild`	Forces a full catalog rebuild, then verifies it actually happened. See the gotcha below — this one earned its verification logic the hard way.

Plumbing

FUNCTION	WHAT IT DOES
`Use-KorRemoteRegistry`	Runs a scriptblock with RemoteRegistry temporarily available, always restoring it afterwards.
`Invoke-KorSc`	Uniform `sc.exe` wrapper so failures are visible rather than swallowed.
`Get-KorServiceState / Wait-KorServiceState`	Poll for a target service state. Never blind-sleep past a stop — confirm `STOPPED` before touching files.

USAGE

A worked example

The output below is the real result from the machine that started all of this:

```
PS> Get-KorWorkstationHealth -ComputerName KOR-206-N |
      Select-Object -ExpandProperty Findings

Office shared-DLL skew: down-level 16.0.5023.1000 (2020-07-14)
  vs C2R 16.0.20228.20110 — owned by Access Database Engine,
  update it, do not delete
Search indexing failing for kevinw\kevinw@korstructural.com.ost
Duplicate Outlook store kevinw@korstructural.com(2).nst — profile damage
Outlook disabled add-in 'Skype Meeting Add-in' on 2026-07-28:
  This add-in caused Outlook to crash. As a result, it was disabled.
Slow add-in 'EmailFilerv2' costs 546ms every launch
Repeat faulting process 'opushutil.exe' x17 in 14 days
```

Fleet sweep

```
$fleet = 'KOR-206-N','KOR-207','KOR-308','KOR-320'
$fleet | Get-KorWorkstationHealth | Where-Object { $_.Findings }
```

Remediation with rollback

```
PS> Set-KorOutlookAddinState -ComputerName KOR-207 `
      -ProgID UCAddin.LyncAddin.1,UCAddin.UCAddin.1 -LoadBehavior 0

ProgID          : UCAddin.LyncAddin.1
PriorValue      : 3
NewValue        : 0
RollbackCommand : Set-KorOutlookAddinState -ComputerName KOR-207
                    -ProgID UCAddin.LyncAddin.1 -LoadBehavior 3
```

RESULTS

What it found in one evening

Run across the fleet on the night it was written, from a single helpdesk ticket:

- **A fleet-wide Office conflict on 13 of 27 machines.** The Access Database Engine redistributable installs down-level copies of the Office shared DLLs into `Common Files\microsoft shared\Office16`. Click-to-Run Office binds to those instead of its own and crashes. Proven on an 8/8 split: every machine with the Access Engine showed 16–23 `opushutil.exe` delay-load failures per fortnight; every machine without it, on the same Office build, showed zero.
- **KOR-207 — Revit crashing five times in twenty-three minutes**, on the oldest Revit 2026 build in service. Never reported by the user.
- **KOR-308 — Bluebeam crashing fourteen times in a fortnight.** Never reported.
- **Eleven distinct NVIDIA driver versions across 21 machines**, the oldest dating to July 2021.
- **An in-house application crashing nine times in a fortnight** on one machine alone.

THE RECURRING LESSON

Almost none of this was reported by anybody. Users normalise friction — the person with 31 stale mailbox stores and five Revit crashes in one afternoon had not raised a ticket. **The value of a fleet sweep is not diagnosing the machine that complained; it is finding the ones that never will.**

HARD-WON

Gotchas & bugs found by running it

Every item here was discovered by executing the module against live machines, not by reading it. They are recorded because each would otherwise be rediscovered painfully.

THE DANGEROUS ONE — A FIX THAT LIED

Windows Search **silently ignores** the rebuild flag if the service is restarted immediately after the registry write. The flag stays `0`, the old catalog survives untouched, and the operation looks like it worked. The original verification was merely "*did the size go down*", which an 8 MB drift passed — so it reported success on a machine where nothing had happened.

It now polls for the catalog to collapse below half its starting size, cycles the service up to three times, and warns loudly if the rebuild genuinely did not run. **A remediation that reports success without verifying the state changed is worse than one that fails noisily.**

GOTCHA	DETAIL
Add-in state is stale	`Get-KorOutlookAddin` reports what Outlook logged at its *last launch*, not the current registry. After changing LoadBehavior it reads unchanged until the user restarts Outlook. Verify via `reg.exe`, not this function.
Identical store names	Two Windows profiles for the same person hold identically-named OSTs. Attributing index errors by file name double-counts them onto the wrong profile — match the full local path.
Duplicate stores need an age	Abandoned profiles keep their stores forever. Thirty-one duplicates from 2023 read as catastrophic damage until you check the dates and find residue from a problem that resolved three years ago. Findings now carry `AgeDays` and split LIVE from CLEANUP.
UNC globs	`-Include` does not resolve against a UNC path with a mid-path wildcard. Glob each extension directly.
RemoteRegistry cost	Each start/stop cycle is slow over VPN. Batch registry work into one `Use-KorRemoteRegistry` block rather than one per item.

BOUNDARIES

What it cannot do

Stated plainly, so time is not wasted trying:

- **Enumerate shared calendars.** Subscriptions are binary MAPI blobs inside the OST. They appear in no registry string, no event log and no disk artifact. A stale subscription to a room mailbox deleted in 2021 cost one user seconds on every calendar open for five years and left *no* remotely detectable trace. Finding these means asking users or looking at the calendar list in Outlook.
- **Read a logged-in user's hive offline.** `NTUSER.DAT` is locked while the user is logged on. Read it live through `HKU` instead, which requires them to be logged in — the opposite constraint.
- **Analyse crash dumps.** A recurring `ucrtbase.dll` `0xc0000409` fail-fast can be counted and correlated but not explained without dump analysis, which needs local access.

- **Anything interactive.** Rebuilding an Outlook profile, installing software, or confirming a UI is actually fast all require a human at the machine.

> **IF WINRM EVER LANDS**
>
> Nothing here depends on WinRM staying blocked. Every function keeps working if the scoped GPO is applied — event queries simply stop needing a 20 MB file copy per machine, which matters most when administering over the VPN.

KOR Structural · WorkstationOps toolkit · `tools/WorkstationOps/`
Prepared 2026-07-28 · validated against 9 live machines · companion document: *KOR-Workstation-Office-ACE-Conflict-2026-07-28*